

GitHub Copilot Hackathon

Objective

Experience how GitHub Copilot can accelerate modern software development by building a simple, end-to-end application within a few hours.

Participants will use Copilot to:

- Rapidly scaffold application components (frontend + API)
- Implement core functionality using natural language prompts
- Apply basic business logic or validation to real-world scenarios

Theme: Document Validation

Build an app that validates documents such as HR forms, invoices, certificates, or ID documents.

The goal is not to build a production-ready system, but to demonstrate how AI-assisted development can significantly reduce effort, improve productivity, and enable faster innovation.

Participants

- Work individually, with optional pairing if preferred
- Optional: work in a GitHub repository to collect artifacts after the event

Pre-Requisites (Send 3+ Days Before)

Participants must have the following ready before the hackathon day:

- VS Code installed
- GitHub Copilot extension installed and licensed
- Git installed
- Runtime of choice installed (Node.js 20+, Python 3.11+, Java 17+, or .NET 8+)
- Familiarity with at least one web framework

Agenda

Phase	Duration
Kickoff & Setup	30 min
Build Phase 1 — Core	2 hours
Build Phase 2 — Enhance	1.5 hours
Build Phase 3 — Stretch	1 hour
Demos & Wrap-Up	1 hour

Kickoff & Setup (30 min)

- Welcome and session goals
- Live demo: Build a small feature with Copilot in 5 minutes (shows agent mode, inline completions, chat)
- Overview of the challenge and example use cases
- Verify environments are working

Build Phase 1 — Core Flow (2 hours)

Goal: **Achieve a working end-to-end flow.**

- Create a simple frontend (upload form or input UI)
- Build an API endpoint to receive and process data
- Implement basic data extraction or parsing logic
- Connect frontend → API → response displayed to user

Tip: Use a starter template (see below) to skip boilerplate and jump straight to the interesting parts.

Build Phase 2 — Enhance (1.5 hours)

Goal: **Add business value and polish.**

- Add validation rules or business logic (required fields, date checks, format validation)
- Enhance results display (clear pass/fail indicators, detailed feedback, insights)
- Improve user experience (error handling, loading states, styling)

Build Phase 3 — Stretch (1 hour)

Goal: **Go beyond the basics.** For teams that have a solid working app.

- Add unit tests using Copilot-generated test suggestions
- Support multiple document types — add a second validation scenario (e.g., invoices + HR forms) with different rules
- Accept PDF uploads — extract text from uploaded files instead of form input
- Build a summary dashboard showing all validation results
- Add data persistence (save and retrieve past validations)
- Create a reusable Copilot SKILL file — write a .md skill that captures your validation rules, patterns, or domain knowledge so Copilot can use it in future projects
- Add custom instructions (.github/copilot-instructions.md) — define project-level rules Copilot follows automatically, e.g., consistent error response format or validation patterns
- Build a custom agent (.agent.md) — define a specialized agent with specific instructions, e.g., a “validation expert” that knows your document schema and suggests validation patterns

Demos & Wrap-Up (1 hour)

- Team presentations (5–8 minutes each)
- Share what worked, what surprised you, and favorite Copilot moments
- Feedback survey

Example Use Cases

Pick one to keep scope manageable:

Use Case	What It Validates
Invoice Checker	Required fields present (vendor, amount, date, PO number), amounts are positive, date is not in the future
HR Form Validator	Employee name, ID, department filled in; signature present; dates are logical
ID Document Verifier	Expiry date check, required fields present, format consistency
Certificate Validator	Issuer information present, valid date range, certificate number format
Expense Report Auditor	Line items sum to total, receipts attached, within policy limits

Recommended Starter Stacks

Teams are free to choose their own, but here are four fast-start paths:

Stack	Frontend	Backend	Good For
JavaScript	React or plain HTML	Node.js + Express	JS/TS-comfortable teams
Python	HTML + Jinja or React	Flask / FastAPI	Data/ML-leaning teams
Java	Thymeleaf or React	Spring Boot	Java teams
.NET	Blazor or Razor Pages	ASP.NET Core Web API	.NET teams

What Success Looks Like

- A working application (frontend + API) running locally
- Clear demonstration of Copilot-assisted development during the presentation
- A practical, business-relevant validation scenario
- A creative approach and thoughtful user experience
- A team that learned something new about AI-assisted development